CAREER CODE

Job Portal Website

Technical Documentation

Version 1.0

Author: Md.Mesbahul Alam

GitHub: github.com/mesbahtoha

Email: mesbahulalam017@gmail.com

Live: carrer-code-client.vercel.app

Table of Contents

1. Project Overview 4
2. Key Features 4
   2.1 For Job Seekers 4
   2.2 For Employers / Recruiters 4
   2.3 General 4
3. Technology Stack 4
   3.1 Frontend 4
   3.2 Backend 5
   3.3 Deployment 5
4. Project Structure 5
   4.1 Root Structure 5
   4.2 Frontend Structure 6
   4.3 Backend Structure 6
5. Database Schemas 6
   5.1 User Schema 7
   5.2 Job Schema 7
   5.3 Application Schema 7
6. API Reference 8
   6.1 Authentication Endpoints 8
      POST /auth/register — Request Body 8
      POST /auth/login — Request Body 8
      POST /auth/login — Response 8
   6.2 Job Endpoints 8
   6.3 Application Endpoints 9
7. Installation & Setup Guide 9
   7.1 Prerequisites 9
   7.2 Clone the Repository 9
   7.3 Backend Setup 9
   7.4 Frontend Setup 10
8. Deployment 10
   8.1 Frontend — Vercel 10
   8.2 Backend — Render 10
   8.3 Database — MongoDB Atlas 10
9. Environment Variables 11

9.1 Backend (.env) 11
9.2 Frontend (.env) 11
10. Security Considerations 11
11. Contributing 11
12. Author & Contact 12

1. Project Overview

Career Code is a full-stack Job Portal web application built with the MERN stack (MongoDB, Express.js, React.js, Node.js). It connects job seekers with employers by providing a clean, fast, and intuitive platform for job discovery and recruitment.

The application consists of two main parts:

- carrer-code-client — The React frontend, deployed on Vercel
- carrer-code-server — The Node.js/Express backend API, connected to MongoDB

---

2. Key Features

2.1 For Job Seekers

- Browse and search job listings by title, location, or category
- View detailed job descriptions and requirements
- Apply for jobs with a single click
- Create and manage a personal profile
- Track submitted applications

2.2 For Employers / Recruiters

- Post new job listings with full details
- Manage, update, and delete existing job posts
- View applicants for each job posting

2.3 General

- User registration and login with secure JWT authentication
- Role-based access control — Job Seeker vs Employer
- Fully responsive design for desktop and mobile
- Fast and optimized REST API

---

3. Technology Stack

3.1 Frontend

Technology	Version	Purpose
React.js	18+	UI Library

React Router DOM	6+	Client-side routing
Tailwind CSS	3+	Styling and responsive design
Axios	Latest	HTTP requests to backend
Context API	Built-in	Global state management

3.2 Backend

Technology	Version	Purpose
Node.js	18+	JavaScript runtime environment
Express.js	4+	Web application framework
MongoDB	Latest	NoSQL database
Mongoose	7+	ODM for MongoDB
JWT (jsonwebtoken)	Latest	Stateless authentication
bcrypt.js	Latest	Password hashing
dotenv	Latest	Environment variable management
CORS	Latest	Cross-origin resource sharing

3.3 Deployment

Service	Purpose
Vercel	Frontend hosting
Render / Railway	Backend hosting
MongoDB Atlas	Cloud database

4. Project Structure

4.1 Root Structure

```
Job-Portal-Website/
```

```
├── carrer-code-client/     # React frontend
├── carrer-code-server/     # Node.js backend
└── README.md
```

4.2 Frontend Structure

```
carrer-code-client/
├── public/
├── src/
│   ├── assets/             # Images, icons
│   ├── components/         # Reusable UI components
│   ├── pages/              # Route-level page components
│   ├── context/            # Auth context & global state
│   ├── hooks/              # Custom React hooks
│   ├── utils/              # Helper functions
│   ├── App.jsx             # Root component with routes
│   └── main.jsx            # Entry point
├── .env                    # Environment variables
└── package.json
```

4.3 Backend Structure

```
carrer-code-server/
├── config/
│   └── db.js               # MongoDB connection
├── controllers/
│   ├── authController.js   # Register/Login logic
│   ├── jobController.js    # Job CRUD logic
│   └── applicationController.js
├── middleware/
│   ├── authMiddleware.js   # JWT verification
│   └── errorMiddleware.js  # Global error handler
├── models/
│   ├── User.js             # User schema
│   ├── Job.js              # Job schema
│   └── Application.js      # Application schema
├── routes/
│   ├── authRoutes.js
│   ├── jobRoutes.js
│   └── applicationRoutes.js
├── .env
├── index.js                # Express app entry
└── package.json
```

5. Database Schemas

5.1 User Schema

Field	Type	Required	Description
name	String	Yes	Full name of the user
email	String	Yes	Unique email address
password	String	Yes	Hashed password (bcrypt)
role	String (enum)	Yes	'jobseeker' or 'employer'
createdAt	Date	Auto	Account creation timestamp

5.2 Job Schema

Field	Type	Required	Description
title	String	Yes	Job title
company	String	Yes	Company name
location	String	Yes	Job location
description	String	Yes	Full job description
requirements	String	No	Skills and requirements
salary	String	No	Salary range or amount
jobType	String	No	Full-time, Part-time, Remote
postedBy	ObjectId (ref: User)	Yes	Employer who posted the job
createdAt	Date	Auto	Job posting timestamp

5.3 Application Schema

Field	Type	Required	Description
job	ObjectId (ref: Job)	Yes	Reference to the job
applicant	ObjectId (ref: User)	Yes	Reference to the job

			seeker
status	String (enum)	No	pending / reviewed / accepted / rejected
appliedAt	Date	Auto	Submission timestamp

6. API Reference

Base URL (Development): http://localhost:5000/api

Base URL (Production): https://your-backend-url.com/api

6.1 Authentication Endpoints

Method	Endpoint	Auth Required	Description
POST	/auth/register	No	Register a new user account
POST	/auth/login	No	Login and receive JWT token

POST /auth/register — Request Body

```
{ "name": "John Doe", "email": "john@example.com", "password": "secret123",
"role": "jobseeker" }
```

POST /auth/login — Request Body

```
{ "email": "john@example.com", "password": "secret123" }
```

POST /auth/login — Response

```
{ "token": "<JWT_TOKEN>", "user": { "_id": "...", "name": "John Doe", "role":
"jobseeker" } }
```

6.2 Job Endpoints

Method	Endpoint	Auth Required	Description
GET	/jobs	No	Fetch all job listings
GET	/jobs/:id	No	Fetch a single job by

			ID
POST	/jobs	Yes (Employer)	Create a new job posting
PUT	/jobs/:id	Yes (Employer)	Update a job posting
DELETE	/jobs/:id	Yes (Employer)	Delete a job posting

6.3 Application Endpoints

Method	Endpoint	Auth Required	Description
POST	/applications	Yes (Job Seeker)	Apply for a job
GET	/applications/me	Yes	Get current user's applications
GET	/applications/job/:id	Yes (Employer)	Get all applicants for a job

Note: All protected endpoints require Authorization header:

```
Authorization: Bearer <JWT_TOKEN>
```

---

7. Installation & Setup Guide

7.1 Prerequisites

- Node.js v18 or higher
- npm v9 or higher
- MongoDB (local install or MongoDB Atlas cloud account)
- Git

7.2 Clone the Repository

```
git clone https://github.com/mesbahtoha/Job-Portal-Website.git
cd Job-Portal-Website
```

7.3 Backend Setup

Step 1: Navigate to the server folder and install dependencies:

```
cd carrer-code-server
npm install
```

Step 2: Create a .env file in carrer-code-server/:

```
PORT=5000
MONGO_URI=mongodb+srv://<username>:<password>@cluster.mongodb.net/careercode
JWT_SECRET=your_super_secret_key_here
```

Step 3: Start the backend server:

```
npm start          # production
npm run dev        # development with nodemon
```

The server will be available at: http://localhost:5000

7.4 Frontend Setup

Step 1: Navigate to the client folder and install dependencies:

```
cd ../carrer-code-client
npm install
```

Step 2: Create a .env file in carrer-code-client/:

```
VITE_API_URL=http://localhost:5000/api
```

Step 3: Start the frontend development server:

```
npm run dev
```

The app will be available at: http://localhost:5173

---

8. Deployment

8.1 Frontend — Vercel

- Go to vercel.com and sign in with GitHub
- Click 'New Project' and import the Job-Portal-Website repo
- Set the root directory to carrer-code-client
- Add environment variable: VITE_API_URL = your backend URL
- Click Deploy

8.2 Backend — Render

- Go to render.com and create a new Web Service
- Connect your GitHub repo
- Set root directory to carrer-code-server
- Set build command to: npm install
- Set start command to: node index.js
- Add all environment variables (PORT, MONGO_URI, JWT_SECRET)
- Deploy

8.3 Database — MongoDB Atlas

- Create a free cluster at mongodb.com/cloud/atlas

- Create a database user with read/write permissions
- Whitelist your server's IP (or use 0.0.0.0/0 for all IPs)
- Copy the connection string and paste it into MONGO_URI in your .env

9. Environment Variables

9.1 Backend (.env)

Variable	Description	Example
PORT	Port for the Express server	5000
MONGO_URI	MongoDB connection string	mongodb+srv://...
JWT_SECRET	Secret key for signing JWTs	mySecretKey123

9.2 Frontend (.env)

Variable	Description	Example
VITE_API_URL	Backend API base URL	http://localhost:5000/api

10. Security Considerations

- Passwords are hashed with bcrypt before storing — never stored in plain text
- JWTs are signed with a secret key and expire after a set duration
- Protected routes are verified using JWT middleware on the backend
- Role-based access control ensures only employers can post/update/delete jobs
- Environment variables are used for all sensitive config — never commit .env files
- CORS is configured to allow requests only from trusted frontend origins

11. Contributing

Contributions are welcome! To contribute to this project:

1. Fork the repository on GitHub

2. Create a feature branch: git checkout -b feature/my-feature
3. Commit your changes: git commit -m "Add my feature"
4. Push to the branch: git push origin feature/my-feature
5. Open a Pull Request on GitHub

12. Author & Contact

Field	Details
Name	Md.Mesbahul Alam
GitHub	github.com/mesbahtoha
Email	mesbahulalam017@gmail.com
Project Repo	github.com/mesbahtoha/Job-Portal-Website
Live Demo	carrer-code-client.vercel.app
Stack	MERN (MongoDB, Express, React, Node.js)

CarrerCode

Job Portal Website

Developed by **Md.Mesbahul Alam**

github.com/mesbahtoha | mesbahulalam017@gmail.com